

2a. Gemini 3.8 Flash Extended, Qwen3.6 35B A3B with Reasoning

2b. Gemini via <https://gemini.google.com/app>; Qwen via LM Studio

2c.

Implement a C program based on these specifications, output a single C file for the entire program:

- * The program will prioritize emails for a busy company CEO.

- * You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.

- * You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.

- * You must create functions from scratch. Do not include pre-existing heap modules.

Test File format:

Command | Description

EMAIL <sender category>,

<subject line>,

<date> |

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces,

but not commas

Assume <date> is in the format: MM-DD-YYYY |

NEXT |

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date: |

READ |

CEO has read email and dealt with it |

COUNT |

Display current count of unread emails

* The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

* Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

* Your program should display this:

There are 6 emails to read.

Next email:

\t Sender: Boss

\t Subject: Never Mind

\t Date: 01-03-2025

Next email:

\t Sender: Boss

\t Subject: Important

\t Date: 12-20-2024

There are 4 emails to read.

2d. Gemini:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5
6  #define MAX_SENDER_LEN 64
7  #define MAX_SUBJECT_LEN 256
8  #define MAX_DATE_LEN 32
9  #define INITIAL_CAPACITY 16
10
11 typedef struct {
12     int month;
13     int day;
14     int year;
15 } Date;
16
17 typedef struct {
18     char sender[MAX_SENDER_LEN];
19     char subject[MAX_SUBJECT_LEN];
20     char dateStr[MAX_DATE_LEN];
21     Date date;
22     int order;
23 } Email;
24
25 typedef struct {
26     Email *array;
27     int capacity;
28     int size;
29 } MaxHeap;
30
31 /* Helper string trimming function */
32 static void trim(char *str) {
33     if (!str) return;
34
35     char *start = str;
36     while (*start && isspace((unsigned char)*start)) {
37         start++;
38     }
39     if (start != str) {
40         memmove(str, start, strlen(start) + 1);
41     }
42
43     int len = (int)strlen(str);
44     while (len > 0 && isspace((unsigned char)str[len - 1])) {
45         str[len - 1] = '\0';
46         len--;
47     }
48 }
49
50 /* Parse date from format MM-DD-YYYY */
51 static Date parseDate(const char *dateStr) {
52     Date d = {0, 0, 0};
53     sscanf(dateStr, "%d-%d-%d", &d.month, &d.day, &d.year);
54     return d;
55 }
56
57 /* Return numeric priority for sender categories (higher number = higher priority) */
58 static int getCategoryPriority(const char *category) {
59     if (strcmp(category, "Boss") == 0) return 5;
60     if (strcmp(category, "Subordinate") == 0) return 4;
61     if (strcmp(category, "Peer") == 0) return 3;
62     if (strcmp(category, "ImportantPerson") == 0) return 2;
63     if (strcmp(category, "OtherPerson") == 0) return 1;
64     return 0;
65 }
66
67 /* Compare dates: returns > 0 if d1 is newer than d2, < 0 if older, 0 if identical */
68 static int compareDates(Date d1, Date d2) {
69     if (d1.year != d2.year) {
70         return d1.year - d2.year;
71     }
72     if (d1.month != d2.month) {
73         return d1.month - d2.month;
74     }
75     return d1.day - d2.day;
76 }
77
78 /* Compare two emails: returns > 0 if a has higher priority than b, < 0 if lower */
79 static int compareEmails(const Email *a, const Email *b) {
80     int priA = getCategoryPriority(a->sender);
81     int priB = getCategoryPriority(b->sender);

```

```

82
83     if (priA != priB) {
84         return priA - priB;
85     }
86
87     /* Tie-breaker: newest email first */
88     int dateComparison = compareDates(a->date, b->date);
89     if (dateComparison != 0) {
90         return dateComparison;
91     }
92
93     /* Secondary tie-breaker: preserve earlier arrival */
94     return b->order - a->order;
95 }
96
97 /* Create and initialize an empty MaxHeap */
98 MaxHeap* createHeap(int initialCapacity) {
99     MaxHeap *heap = (MaxHeap *)malloc(sizeof(MaxHeap));
100     if (!heap) {
101         perror("Failed to allocate memory for heap");
102         exit(EXIT_FAILURE);
103     }
104     heap->capacity = initialCapacity > 0 ? initialCapacity : INITIAL_CAPACITY;
105     heap->size = 0;
106     heap->array = (Email *)malloc(sizeof(Email) * heap->capacity);
107     if (!heap->array) {
108         perror("Failed to allocate memory for heap array");
109         free(heap);
110         exit(EXIT_FAILURE);
111     }
112     return heap;
113 }
114
115 /* Swap two Email elements */
116 static void swapEmails(Email *a, Email *b) {
117     Email temp = *a;
118     *a = *b;
119     *b = temp;
120 }
121
122 /* Sift up an element to maintain the max-heap property */
123 void heapifyUp(MaxHeap *heap, int index) {
124     while (index > 0) {
125         int parent = (index - 1) / 2;
126         if (compareEmails(&heap->array[index], &heap->array[parent]) > 0) {
127             swapEmails(&heap->array[index], &heap->array[parent]);
128             index = parent;
129         } else {
130             break;
131         }
132     }
133 }
134
135 /* Sift down an element to maintain the max-heap property */
136 void heapifyDown(MaxHeap *heap, int index) {
137     while (1) {
138         int left = 2 * index + 1;
139         int right = 2 * index + 2;
140         int largest = index;
141
142         if (left < heap->size && compareEmails(&heap->array[left], &heap->array[largest]) > 0) {
143             largest = left;
144         }
145         if (right < heap->size && compareEmails(&heap->array[right], &heap->array[largest]) > 0) {
146             largest = right;
147         }
148
149         if (largest != index) {
150             swapEmails(&heap->array[index], &heap->array[largest]);
151             index = largest;
152         } else {
153             break;
154         }
155     }
156 }

```

```

162     Email *newArray = (Email *)realloc(heap->array, sizeof(Email) * heap->capacity);
163     if (!newArray) {
164         perror("Failed to resize heap array");
165         exit(EXIT_FAILURE);
166     }
167     heap->array = newArray;
168 }
169
170 heap->array[heap->size] = email;
171 heapifyUp(heap, heap->size);
172 heap->size++;
173 }
174
175 /* Inspect the highest priority email without removing it */
176 int peekHeap(const MaxHeap *heap, Email *result) {
177     if (heap->size == 0) {
178         return 0;
179     }
180     if (result) {
181         *result = heap->array[0];
182     }
183     return 1;
184 }
185
186 /* Remove and return the highest priority email */
187 int popHeap(MaxHeap *heap, Email *result) {
188     if (heap->size == 0) {
189         return 0;
190     }
191     if (result) {
192         *result = heap->array[0];
193     }
194
195     heap->array[0] = heap->array[heap->size - 1];
196     heap->size--;
197
198     if (heap->size > 0) {
199         heapifyDown(heap, 0);
200     }
201     return 1;
202 }
203
204 /* Free all memory associated with the MaxHeap */
205 void freeHeap(MaxHeap *heap) {
206     if (heap) {
207         if (heap->array) {
208             free(heap->array);
209         }
210         free(heap);
211     }
212 }
213
214 int main(int argc, char *argv[]) {
215     FILE *fp = stdin;
216     if (argc > 1) {
217         fp = fopen(argv[1], "r");
218         if (!fp) {
219             perror("Error opening input file");
220             return EXIT_FAILURE;
221         }
222     }
223
224     MaxHeap *heap = createHeap(INITIAL_CAPACITY);
225     char line[1024];
226     int arrivalOrder = 0;
227
228     while (fgets(line, sizeof(line), fp)) {
229         line[strcspn(line, "\r\n")] = '\0';
230
231         char *cmd = line;
232         while (*cmd && isspace((unsigned char)*cmd)) {
233             cmd++;
234         }
235         if (*cmd == '\0') {
236             continue;

```

```

241     while (*args == ' ' || *args == '\t') {
242         args++;
243     }
244
245     char *comma1 = strchr(args, ',');
246     if (!comma1) continue;
247     *comma1 = '\0';
248
249     char *comma2 = strchr(comma1 + 1, ',');
250     if (!comma2) continue;
251     *comma2 = '\0';
252
253     char *sender = args;
254     char *subject = comma1 + 1;
255     char *dateStr = comma2 + 1;
256
257     trim(sender);
258     trim(subject);
259     trim(dateStr);
260
261     Email email;
262     strncpy(email.sender, sender, sizeof(email.sender) - 1);
263     email.sender[sizeof(email.sender) - 1] = '\0';
264
265     strncpy(email.subject, subject, sizeof(email.subject) - 1);
266     email.subject[sizeof(email.subject) - 1] = '\0';
267
268     strncpy(email.dateStr, dateStr, sizeof(email.dateStr) - 1);
269     email.dateStr[sizeof(email.dateStr) - 1] = '\0';
270
271     email.date = parseDate(email.dateStr);
272     email.order = arrivalOrder++;
273
274     insertHeap(heap, email);
275 } else if (strcmp(cmd, "COUNT") == 0) {
276     printf("There are %d emails to read.\n", heap->size);
277 } else if (strcmp(cmd, "NEXT") == 0) {
278     Email nextEmail;
279     if (peekHeap(heap, &nextEmail)) {
280         printf("Next email:\n");
281         printf("\t Sender: %s\n", nextEmail.sender);
282         printf("\t Subject: %s\n", nextEmail.subject);
283         printf("\t Date: %s\n", nextEmail.dateStr);
284     }
285 } else if (strcmp(cmd, "READ") == 0) {
286     popHeap(heap, NULL);
287 }
288 }
289
290 freeHeap(heap);
291 if (fp != stdin) {
292     fclose(fp);
293 }
294
295 return EXIT_SUCCESS;
296 }

```

Qwen:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <stdbool.h>
5
6  typedef struct {
7      char *sender;
8      char *subject;
9      char *date;
10 } Email;
11
12 typedef struct {
13     Email *data;
14     int size;
15     int capacity;
16 } MaxHeap;
17
18 void init_heap(MaxHeap* h) {
19     h->capacity = 16;
20     h->size = 0;
21     h->data = (Email*)malloc(h->capacity * sizeof(Email));
22 }
23
24 void ensure_capacity(MaxHeap* h) {
25     if (h->size >= h->capacity) {
26         h->capacity *= 2;
27         h->data = (Email*)realloc(h->data, h->capacity * sizeof(Email));
28     }
29 }
30
31 int get_priority(const char* sender) {
32     if (strcmp(sender, "Boss") == 0) return 5;
33     if (strcmp(sender, "Subordinate") == 0) return 4;
34     if (strcmp(sender, "Peer") == 0) return 3;
35     if (strcmp(sender, "ImportantPerson") == 0) return 2;
36     if (strcmp(sender, "OtherPerson") == 0) return 1;
37     return 0;
38 }
39
40 int date_to_int(const char* d) {
41     int m, day, y;
42     sscanf(d, "%d-%d-%d", &m, &day, &y);
43     return y * 10000 + m * 100 + day;
44 }
45
46 bool is_higher_priority(const Email* a, const Email* b) {
47     int p_a = get_priority(a->sender);
48     int p_b = get_priority(b->sender);
49     if (p_a != p_b) return p_a > p_b;
50     int d_a = date_to_int(a->date);
51     int d_b = date_to_int(b->date);
52     return d_a > d_b; // newer first
53 }
54
55 void swap_emails(Email* a, Email* b) {
56     Email temp = *a;
57     *a = *b;
58     *b = temp;
59 }
60
61 void heapify_up(MaxHeap* h, int idx) {
62     while (idx > 0) {
63         int parent = (idx - 1) / 2;
64         if (is_higher_priority(&h->data[idx], &h->data[parent])) {
65             swap_emails(&h->data[idx], &h->data[parent]);
66             idx = parent;
67         } else break;
68     }
69 }
70
71 void heapify_down(MaxHeap* h, int idx) {
72     while (true) {
73         int largest = idx;
74         int left = 2 * idx + 1;
75         int right = 2 * idx + 2;
76
77         if (left < h->size & is_higher_priority(&h->data[left], &h->data[largest])) largest = left;
78         if (right < h->size & is_higher_priority(&h->data[right], &h->data[largest])) largest = right;
79         if (largest != idx) {
80             swap_emails(&h->data[idx], &h->data[largest]);
81             idx = largest;
82         } else break;
83     }
84 }
85
86 void insert(MaxHeap* h, Email* e) {
87     ensure_capacity(h);
88     h->data[h->size] = *e;
89     heapify_up(h, h->size);
90     h->size++;
91 }
92
93 void delete(MaxHeap* h, int idx) {
94     if (idx < 0 || idx >= h->size) return;
95     swap_emails(&h->data[idx], &h->data[h->size - 1]);
96     heapify_down(h, idx);
97     h->size--;
98 }
99
100 void print_heap(MaxHeap* h) {
101     for (int i = 0; i < h->size; i++) {
102         printf("%d: %s, %s, %s\n", i, h->data[i].sender, h->data[i].subject, h->data[i].date);
103     }
104 }
105
106 int main() {
107     MaxHeap h;
108     init_heap(&h);
109
110     Email e1 = {"Boss", "Meeting", "2023-10-26"};
111     Email e2 = {"Subordinate", "Report", "2023-10-25"};
112     Email e3 = {"Peer", "Discussion", "2023-10-24"};
113     Email e4 = {"ImportantPerson", "Urgent", "2023-10-23"};
114     Email e5 = {"OtherPerson", "Casual", "2023-10-22"};
115
116     insert(&h, &e1);
117     insert(&h, &e2);
118     insert(&h, &e3);
119     insert(&h, &e4);
120     insert(&h, &e5);
121
122     print_heap(&h);
123
124     delete(&h, 0);
125     print_heap(&h);
126
127     return 0;
128 }

```

```

82     if (largest != idx) {
83         swap_emails(&h->data[idx], &h->data[largest]);
84         idx = largest;
85     } else break;
86 }
87 }
88
89 void insert(MaxHeap* h, Email e) {
90     ensure_capacity(h);
91     h->data[h->size] = e;
92     heapify_up(h, h->size);
93     h->size++;
94 }
95
96 Email extract_max(MaxHeap* h) {
97     if (h->size == 0) return (Email){NULL, NULL, NULL};
98     Email top = h->data[0];
99     h->size--;
100    if (h->size > 0) {
101        h->data[0] = h->data[h->size];
102        heapify_down(h, 0);
103    }
104    return top;
105 }
106
107 int get_size(MaxHeap* h) { return h->size; }
108
109 void free_email(Email e) {
110     if (e.sender) free(e.sender);
111     if (e.subject) free(e.subject);
112     if (e.date) free(e.date);
113 }
114
115 int main() {
116     MaxHeap heap;
117     init_heap(&heap);
118
119     char line[1024];
120     while (fgets(line, sizeof(line), stdin)) {
121         // Remove trailing newline/carriage return
122         size_t len = strlen(line);
123         while (len > 0 && (line[len-1] == '\n' || line[len-1] == '\r')) {
124             line[--len] = '\0';
125         }
126
127         if (strcmp(line, "EMAIL ", 6) == 0) {
128             char *rest = line + 6;
129             char *c1 = strchr(rest, ',');
130             if (!c1) continue; // malformed
131
132             size_t sender_len = c1 - rest;
133             char *sender_str = (char*)malloc(sender_len + 1);
134             strncpy(sender_str, rest, sender_len);
135             sender_str[sender_len] = '\0';
136
137             char *sub_start = c1 + 1;
138             char *c2 = strchr(sub_start, ',');
139             if (!c2) continue; // malformed
140
141             size_t sub_len = c2 - sub_start;
142             char *subject_str = (char*)malloc(sub_len + 1);
143             strncpy(subject_str, sub_start, sub_len);
144             subject_str[sub_len] = '\0';
145
146             char *date_str = strdup(c2 + 1); // strdup is POSIX but widely available. Fallback if needed? I'll use malloc-strncpy for strict C.
147             size_t date_len = strlen(c2 + 1);
148             date_str = (char*)malloc(date_len + 1);
149             strcpy(date_str, c2 + 1);
150
151             Email e = {sender_str, subject_str, date_str};
152             insert(&heap, e);
153         } else if (strcmp(line, "NEXT") == 0) {
154             if (get_size(&heap) > 0) {
155                 printf("Next email:\n");
156                 printf("\t Sender: %s\n", heap.data[0].sender);
157                 printf("\t Subject: %s\n", heap.data[0].subject);
158                 printf("\t Date: %s\n", heap.data[0].date);
159             }
160         }
161     }
162 }

```

```

160         } else if (strcmp(line, "READ") == 0) {
161             if (get_size(&heap) > 0) {
162                 Email e = extract_max(&heap);
163                 free_email(e);
164             }
165         } else if (strcmp(line, "COUNT") == 0) {
166             printf("There are %d emails to read.\n", get_size(&heap));
167         }
168     }
169
170     // Cleanup remaining heap
171     for (int i = 0; i < heap.size; i++) free_email(heap.data[i]);
172     free(heap.data);
173
174     return 0;
175 }

```

2e. Gemini:

- a. Correctness: Handles the happy path correctly. When I put commands in a bad order into the file, it handles them as appropriate.
- b. Execution Speed: Everytime an email needs to be checked for its priority, 5 string comparisons are executed, when they could be stored in the email struct, since they do not change. Email structs are also moved around in the heap in memory, moving all of their data, versus storing every email in a heap, and then sorting their pointers, which are cheaper to move.
- c. Space Complexity: The date is stored twice in an email in two different formats, and the MAX_SENDER_LEN is overly large for the maximum number of characters that a sender has. The sender is also stored as a string in the email, when the types of senders are static.
- d. Maintainability: Uses preprocessor constants to enable quick modifications while in-lining those changes. Every function and variable is named appropriately to know its purpose by reading it. Category priority parsing is hardcoded in a function in the middle of the program, which could make it harder to later find and update with new categories. The MaxHeap's internal array is also directly exposed, which means any changes to how it stores data requires changes everywhere else. Bad inputs are just ignored silently with no error or warning.

Qwen:

- a. Correctness: Handles the happy path correctly and handles basic edge cases.

- b. Execution Speed: Everytime an email needs to be checked for its priority, this also executes 5 string comparisons. The dates are also converted into an integer every time there is an email comparison, rather than just storing the converted date in the Email struct. In the main function when getting date_str, strdup is called unnecessarily because it is immediately overridden.
- c. Space Complexity: The call to strdup in the date string parsing of the EMAIL block leaks memory for every email, because the pointer to the new string is immediately lost. The sender is stored as a string when the kind of sender is static.
- d. Maintainability: Lots of variables are hard-coded, rather than using constants/preprocessor definitions. Malformed input is ignored silently. The main function also handles most of the logic, rather than being separated out into functions. Numerous variables are also named without context as to what they do, which makes the code harder to read.

2f. I choose Gemini, because the code generated is safer in terms of the error handling it does have and because of the strong maintainability that can be quickly improved further.

2g. To improve correctness, I added an error check when parsing the date string for an email. To improve execution speed and maintainability, I converted the sender categories into an enum so comparisons didn't have to make conversions (I added a function to parse an enum into a proper string for printing). I removed the dateStr field from Email and changed the sender field to be sender_category, which is just an integer from the Category enum. To improve maintainability, I moved the email parser into a separate function, reducing the bulk in the main function. To account for the changes in data types for Email, I also modified the NEXT command block to parse appropriately.